Chapter 20. Policy and Governance for Kubernetes Clusters

Throughout this book we have introduced many different Kubernetes resource types, each with a specific purpose. It doesn't take long before the resources on a Kubernetes cluster go from several, for a single microservice application, to hundreds and thousands, for a complete distributed application. In the context of a production cluster it isn't hard to imagine the challenges associated with managing thousands of resources.

In this chapter, we introduce the concepts of policy and governance. *Policy* is a set of constraints and conditions for how Kubernetes resources can be configured. *Governance* provides the ability to verify and enforce organizational policies for all resources deployed to a Kubernetes cluster, such as ensuring all resources use current best practices, comply with security policy, or adhere to company conventions. Whatever your case may be, your tooling needs to be flexible and scalable so that all resources defined on a cluster comply with your organization's defined policies.

Why Policy and Governance Matter

There are many different types of policies in Kubernetes. For example, NetworkPolicy allows you to specify what network services and endpoints a Pod can connect to. PodSecurityPolicy enables fine-grained control over the security elements of a Pod. Both can be used to configure network or container runtimes.

However, you might want to enforce a policy before Kubernetes resources are even created. This is the problem that policy and governance solve. At this point, you might be thinking, "Isn't this what role-based access control does?" However, as you'll see in this chapter, RBAC isn't granular enough to restrict specific fields within resources from being set.

Here are some common examples of policies that cluster administrators often configure:

- All containers *must* only come from a specific container registry.
- All Pods *must* be labeled with the department name and contact information.

- All Pods *must* have both CPU and memory resource limits set.
- All Ingress hostnames *must* be unique across a cluster.
- A certain service *must* not be made available on the internet.
- Containers *must* not listen on privileged ports.

Cluster administrators may also want to audit existing resources on a cluster, perform dry-run policy evaluations, or even mutate a resource based on a set of conditions—for example, applying labels to a Pod if they aren't present.

It's very important for cluster administrators to be able to define policy and perform compliance audits without interfering with the developers' ability to deploy applications to Kubernetes. If developers are creating noncompliant resources, you need a system to make sure they get the feedback and remediation they need to bring their work into compliance.

Let's take a look at how to achieve policy and governance by leveraging core extensibility components of Kubernetes.

Admission Flow

To understand how policy and governance ensures resources are compliant before they are created in your Kubernetes cluster, you must first understand the request flow through the Kubernetes API server. Figure 20-1 depicts the flow of an API request through the API server. Here, we'll focus on mutating admission, validating admission, and webhooks.

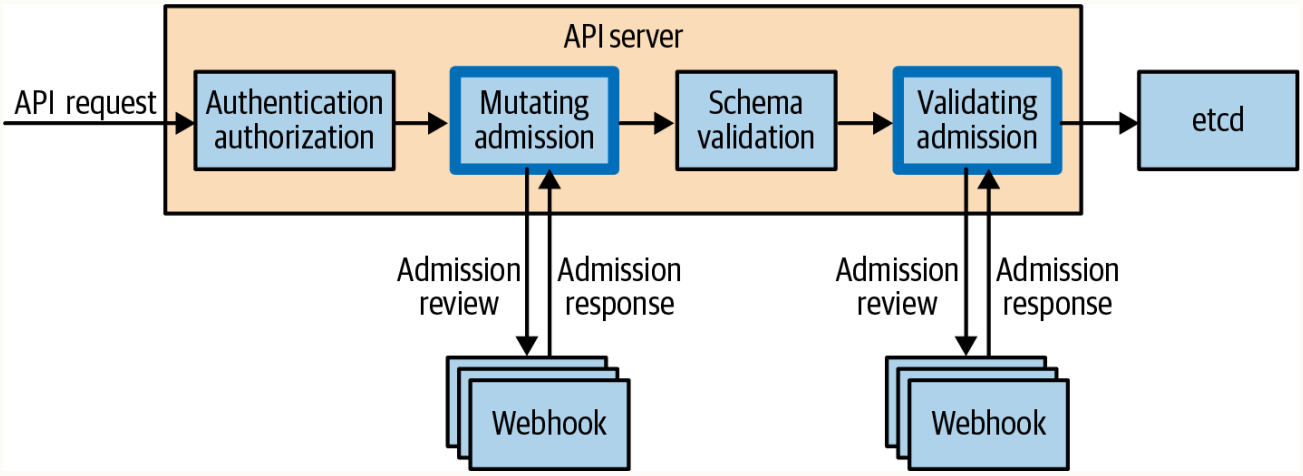


Figure 20-1. API request flow through the Kubernetes API server

Admission controllers operate inline as an API request flows through the Kubernetes API server and are used to either mutate or validate the API request resource before it's

saved to storage. Mutating admission controllers allow the resource to be modified; validating admission controllers do not. There are many different types of admission controllers; this chapter focuses on admission webhooks, which are dynamically configurable. They allow a cluster administrator to configure an endpoint to which the API server can send requests for evaluation by creating either a MutatingWebhookConfiguration or a ValidatingWebhookConfiguration resource. The admission webhook will respond with an "admit" or "deny" directive to let the API server know whether to save the resource to storage.

Policy and Governance with Gatekeeper

Let's dive into how to configure policies and ensure that Kubernetes resources are compliant. The Kubernetes project doesn't provide any controllers that enable policy and governance, but there are open source solutions. Here, we will focus on an open source ecosystem project called Gatekeeper.

Gatekeeper is a Kubernetes-native policy controller that evaluates resources based on defined policy and determines whether to allow a Kubernetes resource to be created or modified. These evaluations happen server-side as the API request flows through the Kubernetes API server, which means each cluster has a single point of processing. Processing the policy evaluations server-side means that you can install Gatekeeper on existing Kubernetes clusters without changing developer tooling, workflows, or continuous delivery pipelines.

Gatekeeper uses *custom resource definitions* (CRDs) to define a new set of Kubernetes resources specific to configuring it, which allows cluster administrators to use familiar tools like `kubectl` to operate Gatekeeper. In addition, it provides real-time, meaningful feedback to the user on why a resource was denied and how to remediate the problem. These Gatekeeper-specific custom resources can be stored in source control and managed using GitOps workflows.

Gatekeeper also performs *resource mutation* (resource modification based on defined conditions) and auditing. It is highly configurable and offers fine-grained control over what resources to evaluate and in which namespaces.

What Is Open Policy Agent?

At the core of Gatekeeper is Open Policy Agent, a cloud native open source policy engine that is extensible and allows policy to be portable across different applications.

Open Policy Agent (OPA) is responsible for performing all policy evaluations and returning either an admit or deny. This gives Gatekeeper access to an ecosystem of policy tooling, such as, Conftest, which enables you to write policy tests and implement them in continuous integration pipelines before deployment.

Open Policy Agent exclusively uses a native query language called Rego for all policies. One of the core tenets of Gatekeeper is to abstract the inner workings of Rego from the cluster administrator and present a structured API in the form of a Kubernetes CRD to create and apply policy. This lets you share parameterized policies across organizations and the community. The Gatekeeper project maintains a policy library solely for this purpose (discussed later in this chapter).

Installing Gatekeeper

Before you start configuring policies, you'll need to install Gatekeeper. Gatekeeper components run as Pods in the `gatekeeper-system` namespace and configure a webhook admission controller.

WARNING

Do not install Gatekeeper on a Kubernetes cluster without first understanding how to safely create and disable policy. You should also review the installation YAML before installing Gatekeeper to ensure that you are comfortable with the resources it creates.

You can install Gatekeeper using the Helm package manager:

```
$ helm repo add gatekeeper https://open-policy-agent.github.io/gatekeeper/charts
$ helm install gatekeeper/gatekeeper --name-template=gatekeeper \
  --namespace gatekeeper-system --create-
```

NOTE

Gatekeeper installation requires cluster-admin permissions and is version specific. Please refer to the official documentation for the latest release of Gatekeeper.

Once the installation is complete, confirm that Gatekeeper is up and running:

```
$ kubectl get pods -n gatekeeper-system
NAME                                             READY   STATUS    RESTARTS   AGE
gatekeeper-audit-54c9759898-ljwp8                1/1     Running   0          1m
gatekeeper-controller-manager-6bcc7f8fb5-4nbkt   1/1     Running   0          1m
gatekeeper-controller-manager-6bcc7f8fb5-d85rn   1/1     Running   0          1m
gatekeeper-controller-manager-6bcc7f8fb5-f8m8j   1/1     Running   0          1m
```

You can also review how the webhook is configured using this command:

```
$ kubectl get validatingwebhookconfiguration -o yaml
apiVersion: admissionregistration.k8s.io/v1
kind: ValidatingWebhookConfiguration
metadata:
  labels:
    gatekeeper.sh/system: "yes"
  name: gatekeeper-validating-webhook-configuration
webhooks:
- admissionReviewVersions:
  - v1
  - v1beta1
  clientConfig:
    service:
      name: gatekeeper-webhook-service
      namespace: gatekeeper-system
      path: /v1/admit
  failurePolicy: Ignore
  matchPolicy: Exact
  name: validation.gatekeeper.sh
  namespaceSelector:
    matchExpressions:
    - key: admission.gatekeeper.sh/ignore
      operator: DoesNotExist
  rules:
  - apiGroups:
    - '*'
    apiVersions:
    - '*'
    operations:
    - CREATE
    - UPDATE
    resources:
    - '*'
  sideEffects: None
```

```
  timeoutSeconds: 3
      ...
```

Under the `rules` section of the output above, we see that all resources are being sent to the webhook admission controller, running as a service named `gatekeeper-webhook-service` in the `gatekeeper-system` namespace. Only resources from namespaces that aren't labeled `admission.gatekeeper.sh/ignore` will be considered for policy evaluation. Finally, the `failurePolicy` is set to `Ignore`, which means that this is a *fail open configuration*: if the Gatekeeper service doesn't respond within the configured timeout of three seconds, the request will be admitted.

Configuring Policies

Now that you have Gatekeeper installed, you can start configuring policies. We will first go through a canonical example and demonstrate how the cluster administrator creates policies. Then we'll look at the developer experience when creating compliant and non-compliant resources. We will then expand on each step to gain a deeper understanding, and walk you through the process of creating a sample policy stating that container images can only come from one specific registry. This example is based on the Gatekeeper policy library.

First, you'll need to configure the policy we need to create a custom resource called a *constraint template*. This is usually done by a cluster administrator. The constraint template in Example 20-1 requires you to provide a list of container repositories as parameters that Kubernetes resources are allowed to use.

Example 20-1. allowedrepos-constraint-template.yaml

```
apiVersion: templates.gatekeeper.sh/v1beta1
kind: ConstraintTemplate
metadata:
  name: k8sallowedrepos
  annotations:
    description: Requires container images to begin with a repo string from a
      specified list.
spec:
  crd:
    spec:
      names:
        kind: K8sAllowedRepos
      validation:
```

```
      # Schema for the `parameters` field
      openAPIV3Schema:
        properties:
          repos:
            type: array
            items:
              type: string
  targets:
    - target: admission.k8s.gatekeeper.sh
      rego: |
        package k8sallowedrepos

        violation[{"msg": msg}] {
          container := input.review.object.spec.containers[_]
          satisfied := [good | repo = input.parameters.repos[_] ; good = starts...
          not any(satisfied)
          msg := sprintf("container <%v> has an invalid image repo <%v>, allowed...
        }

        violation[{"msg": msg}] {
          container := input.review.object.spec.initContainers[_]
          satisfied := [good | repo = input.parameters.repos[_] ; good = starts...
          not any(satisfied)
          msg := sprintf("container <%v> has an invalid image repo <%v>, allowed...)
        }
```

Create the constraint template using the following command:

```
$ kubectl apply -f allowedrepos-constraint-template.yaml
constrainttemplate.templates.gatekeeper.sh/k8sallowedrepos created
```

Now you can create a constraint resource to put the policy into effect (again, playing the role of the cluster administrator). The constraint in Example 20-2 allows all containers with the prefix of `gcr.io/kuar-demo/` in the `default` namespace. The `enforcementAction` is set to "deny": any noncompliant resources will be denied.

Example 20-2. allowedrepos-constraint.yaml

```
apiVersion: constraints.gatekeeper.sh/v1beta1
kind: K8sAllowedRepos
metadata:
  name: repo-is-kuar-demo
spec:
```

```
  enforcementAction: deny
  match:
    kinds:
      - apiGroups: [""]
        kinds: ["Pod"]
    namespaces:
      - "default"
  parameters:
    repos:
      - "gcr.io/kuar-demo/"
```

```
$ kubectl create -f allowedrepos-constraint.yaml
k8sallowedrepos.constraints.gatekeeper.sh/repo-is-kuar-demo created
```

The next step is to create some Pods to test that the policy is indeed working. Example 20-3 creates a Pod using a container image, `gcr.io/kuar-demo/kuard-amd64:blue`, that complies with the constraint we defined in the previous step. Workload resource creation is typically performed by the developer responsible for operating the service or a continuous delivery pipeline.

Example 20-3. compliant-pod.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: kuard
spec:
  containers:
    - image: gcr.io/kuar-demo/kuard-amd64:blue
      name: kuard
      ports:
        - containerPort: 8080
          name: http
          protocol: TCP
```

```
$ kubectl apply -f compliant-pod.yaml
pod/kuard created
```

What happens if we create a noncompliant Pod? Example 20-4 creates a Pod using a container image, `nginx`, that is *not* compliant with the constraint we defined in the pre-

vious step. Workload resource creation would typically be performed by the developer or continuous delivery pipeline responsible for operating the service. Note the output in Example 20-4.

Example 20-4. noncompliant-pod.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx-noncompliant
spec:
  containers:
    - name: nginx
      image: nginx
```

```
$ kubectl apply -f noncompliant-pod.yaml
Error from server ([repo-is-kuar-demo] container <nginx> has an invalid image
repo <nginx>, allowed repos are ["gcr.io/kuar-demo/"]): error when creating
"noncompliant-pod.yaml": admission webhook "validation.gatekeeper.sh" denied
the request: [repo-is-kuar-demo] container <nginx> has an invalid image
repo <nginx>, allowed repos are ["gcr.io/kuar-demo/"]
```

Example 20-4 shows that an error is returned to the user with details on why the resource was not created and how to remediate the issue. Cluster administrators can configure the error message in the constraint template.

NOTE

If your constraint's scope is Pods and you create a resource that generates Pods, such as ReplicaSets, Gatekeeper will return an error. However, it won't be returned to you, the user, but to the controller trying to create the Pod. To see these error messages, look in the event log for the relevant resource.

Understanding Constraint Templates

Now that we have walked through a canoncial example, take a closer look at the constraint template in Example 20-1, which takes a list of container repositories that are allowed in Kubernetes resources.

This constraint template has an `apiVersion` and `kind` that are part of the custom resources used only by Gatekeeper. Under the `spec` section, you'll see the name `K8sAllowedRepos`: remember that name, because you'll use it as the constraint kind when creating constraints. You'll also see a schema that defines an array of strings for the cluster administrator to configure. This is done by providing a list of allowed container registries. It also contains the raw Rego policy definition (under the `target` section). This policy evaluates containers and initContainers to ensure that the container repository name starts with the values provided by the constraint. The `msg` section defines the message that is sent back to the user if the policy is violated.

Creating Constraints

To instantiate a policy, you must create a constraint that provides the template's required parameters. There may be many constraints that match the kind of a specific constraint template. Let's take a closer look at the constraint we used in Example 20-2, which allows only container images that originate from *gcr.io/kuar-demo/*.

You may notice that the constraint is of the `kind` "K8sAllowedRepos," which was defined as part of the constraint template. It also defines an `enforcementAction` of "deny," meaning that noncompliant resources will be denied. `enforcementAction` also accepts "dryrun" and "warn": "dryrun" uses the audit feature to test policies and verify their impact; "warn" sends a warning back to the user with the associated message, but allows them to create or update. The `match` portion defines the scope of this constraint, all Pods in the default namespace. Finally, the `parameters` section is required to satisfy the constraint template (an array of strings). The following demonstrates the user experience when the `enforcementAction` is set to "warn":

```
$ kubectl apply -f noncompliant-pod.yaml
Warning: [repo-is-kuar-demo] container <nginx> has an invalid image repo...
pod/nginx-noncompliant created
```

WARNING

Constraints are only enforced on resource CREATE and UPDATE events. If you already have workloads running on a cluster, Gatekeeper will not reevaluate them until a CREATE or UPDATE event takes place.

Here is a real-world example to demonstrate: say you create a policy that only allows containers from a specific registry. All workloads that are already running on the cluster will continue to do so. If you scale the workload Deployment from 1 to 2, the ReplicaSet will attempt to create another Pod. If that Pod doesn't have a container from an allowed repository, then it will be denied. It's important to set the `enforcementAction` to "dryrun" and audit to confirm that any policy violations are known before setting the `enforcementAction` to "deny."

Audit

Being able to enforce policy on new resources is only one piece of the policy and governance story. Policies often change over time, and you can also use Gatekeeper to confirm that everything currently deployed is still compliant. Additionally, you may already have a cluster full of services and wish to install Gatekeeper to bring these resources into compliance. Gatekeeper's audit capabilities allow cluster administrators to get a list of current, noncompliant resources on a cluster.

To demonstrate how auditing works, let's look at an example. We're going to update the `repo-is-kuar-demo` constraint to have an `enforcementAction` action of "dryrun" (as shown in Example 20-5). This will allow users to create noncompliant resources. We will then determine which resources are noncompliant using audit.

Example 20-5. allowedrepos-constraint-dryrun.yaml

```
apiVersion: constraints.gatekeeper.sh/v1beta1
kind: K8sAllowedRepos
metadata:
  name: repo-is-kuar-demo
spec:
  enforcementAction: dryrun
  match:
    kinds:
      - apiGroups: [""]
        kinds: ["Pod"]
    namespaces:
      - "default"
  parameters:
```

```
    repos:
      - "gcr.io/kuar-demo/"
```

Update the constraint by running the following command:

```
$ kubectl apply -f allowedrepos-constraint-dryrun.yaml
k8sallowedrepos.constraints.gatekeeper.sh/repo-is-kuar-demo configured
```

Create a noncompliant Pod using the following command:

```
$ kubectl apply -f noncompliant-pod.yaml
pod/nginx-noncompliant created
```

To audit the list of noncompliant resources for a given constraint, run a `kubectl get constraint` on that constraint and specify that you want the output in YAML format as follows:

```
$ kubectl get constraint repo-is-kuar-demo -o yaml
apiVersion: constraints.gatekeeper.sh/v1beta1
kind: K8sAllowedRepos
...
spec:
  enforcementAction: dryrun
  match:
    kinds:
    - apiGroups:
      - ""
      kinds:
      - Pod
    namespaces:
    - default
  parameters:
    repos:
    - gcr.io/kuar-demo/
status:
  auditTimestamp: "2021-07-14T20:05:38Z"
        ...
  totalViolations: 1
  violations:
  - enforcementAction: dryrun
    kind: Pod
```

```
    message: container <nginx> has an invalid image repo <nginx>, allowed repos
     are ["gcr.io/kuar-demo/"]
    name: nginx-noncompliant
    namespace: default
```

Under the `status` section, you can see the `auditTimestamp`, which is the last time the audit was run. `totalViolations` lists the number of resources that violate this constraint. The `violations` section lists the violations. We can see that the nginx-noncompliant Pod is in violation and the message with the details why.

NOTE

Using a constraint `enforcementAction` of "dryrun" along with audit is a powerful way to confirm that your policy is having the desired impact. It also creates a workflow to bring resources into compliance.

Mutation

So far we have covered how you can use constraints to validate if a resource is compliant. What about modifying resources to make them compliant? This is handled via the mutation feature in Gatekeeper. Earlier in this chapter, we discussed two different type of admission webhooks, mutating and validating. By default, Gatekeeper is only deployed as a validating admission webhook, but it can be configured to operate as a mutating admission webhook.

NOTE

Mutation features in Gatekeeper are in beta state and may change. We share them to demonstrate Gatekeeper's upcoming capabilities. The installation steps in this chapter do not cover enabling mutation. Please refer to the Gatekeeper project for more information on enabling mutation.

Let's walk through an example to demonstrate the power of mutation. In this example, we will set the `imagePullPolicy` to "Always" on all Pods. We will assume that Gatekeeper is configured correctly to support mutation. Example 20-6 defines a mutation assignment that matches all Pods except those in the "system" namespace, and assigns a value of "Always" to `imagePullPolicy`.

Example 20-6. imagepullpolicyalways-mutation.yaml

```
apiVersion: mutations.gatekeeper.sh/v1alpha1
kind: Assign
metadata:
  name: demo-image-pull-policy
spec:
  applyTo:
  - groups: [""]
    kinds: ["Pod"]
    versions: ["v1"]
  match:
    scope: Namespaced
    kinds:
    - apiGroups: ["*"]
      kinds: ["Pod"]
    excludedNamespaces: ["system"]
  location: "spec.containers[name:*].imagePullPolicy"
  parameters:
    assign:
      value: Always
```

Create the mutation assignment:

```
$ kubectl apply -f imagepullpolicyalways-mutation.yaml
assign.mutations.gatekeeper.sh/demo-image-pull-policy created
```

Now create a Pod. This Pod doesn't have `imagePullPolicy` explicitly set, so by default this field is set to "IfNotPresent." However, we expect Gatekeeper to mutate this field to "Always":

```
$ kubectl apply -f compliant-pod.yaml
pod/kuard created
```

Validate that the `imagePullPolicy` has been successfully mutated to "Always" by running the following:

```
$ kubectl get pods kuard -o=jsonpath="{.spec.containers[0].imagePullPolicy}"

Always
```

NOTE

Mutating admission happens before validating admission, so create constraints that validate the mutations you expect to apply to the specific resource.

Delete the Pod using the following command:

```
$ kubectl delete -f compliant-pod.yaml
pod/kuard deleted
```

Delete the mutation assignment using the following command:

```
$ kubectl delete -f imagepullpolicyalways-mutation.yaml
assign.mutations.gatekeeper.sh/demo-image-pull-policy deleted
```

Unlike validation, mutation provides a way to remediate noncompliant resources automatically on behalf of the cluster administrator.

Data Replication

When writing constraints you may want to compare the value of one field to the value of a field in another resource. A specific example of when you might need to do this is making sure that ingress hostnames are unique across a cluster. By default, Gatekeeper can only evaluate fields within the current resource: if comparisons across resources are required to fulfill a policy, it must be configured. Gatekeeper can be configured to cache specific resources into Open Policy Agent to allow comparisons across resources. The resource in Example 20-7 configures Gatekeeper to cache Namespace and Pod resources.

Example 20-7. config-sync.yaml

```
apiVersion: config.gatekeeper.sh/v1alpha1
kind: Config
metadata:
  name: config
  namespace: "gatekeeper-system"
spec:
  sync:
    syncOnly:
      - group: ""
```

```
      version: "v1"
      kind: "Namespace"
    - group: ""
      version: "v1"
      kind: "Pod"
```

NOTE

You should only cache the specific resources needed to perform a policy evaluation. Having hundreds or thousands of resources cached in OPA will require more memory and may also have security implications.

The constraint template in Example 20-8 demonstrates how to compare something in the Rego section (in this case, unique ingress hostnames). Specifically, "data.inventory" refers to the cache resources, as opposed to "input," which is the resource sent for evaluation from the Kubernetes API server as part of the admission flow. This example is based on the Gatekeeper policy library.

Example 20-8. uniqueingresshost-constraint-template.yaml

```
apiVersion: templates.gatekeeper.sh/v1beta1
kind: ConstraintTemplate
metadata:
  name: k8suniqueingresshost
  annotations:
    description: Requires all Ingress hosts to be unique.
spec:
  crd:
    spec:
      names:
        kind: K8sUniqueIngressHost
  targets:
    - target: admission.k8s.gatekeeper.sh
      rego: |
        package k8suniqueingresshost

        identical(obj, review) {
          obj.metadata.namespace == review.object.metadata.namespace
          obj.metadata.name == review.object.metadata.name
        }

        violation[{"msg": msg}] {
          input.review.kind.kind == "Ingress"
```

```
    re_match("^(extensions|networking.k8s.io)$", input.review.kind.group)
    host := input.review.object.spec.rules[_].host
    other := data.inventory.namespace[ns][otherapiversion]["Ingress"][name]
    re_match("^(extensions|networking.k8s.io)/.+$", otherapiversion)
    other.spec.rules[_].host == host
    not identical(other, input.review)
    msg := sprintf("ingress host conflicts with an existing ingress <%v>"...
  }
```

Data replication is a powerful tool that allows you to make comparisons across Kubernetes resources. We recommend only configuring it if you have policies that require it to function. If you use it, scope it only to the relevant resources.

Metrics

Gatekeeper emits metrics in Prometheus format to enable continuous resource compliance monitoring. You can view simple metrics regarding Gatekeeper's overall health, such as the numbers of constraints, constraint templates, and requests being set to Gatekeeper.

In addition, details on policy compliance and governance are also available:

- The total number of audit violations
- Number of constraints by `enforcementAction`
- Audit duration

NOTE

Completely automating the policy and governance process is the ideal goal, so we strongly recommended that you monitor Gatekeeper from an external monitoring system and set alerts based on resource compliance.

Policy Library

One of the core tenets of the Gatekeeper project is to create reusable policy libraries that can be shared between organizations. Being able to share policies reduces boilerplate policy work and allows cluster administrators to focus on applying policy rather than writing it. The Gatekeeper project has a great policy library. It contains a general library with the most common policies as well as a *pod-security-policy* library that models the capabilities of the PodSecurityPolicy API as Gatekeeper policy. The great thing about

this library is that it is always expanding and is open source, so feel free to contribute any policies that you write.

Summary

In this chapter, you've learned about policy and governance and why they are important as more and more resources are deployed to Kubernetes. We covered the Gatekeeper project, a Kubernetes-native policy controller built on Open Policy Agent, and showed you to use it to meet your policy and governance requirements. From writing policies to auditing, you are now equipped with the know-how to meet your compliance needs.

Previous chapter
19. Securing Applications in Kubernetes

Next chapter
21. Multicluster Application Deployments